

Niyukt V1 — Day 1 Engineering Record

Backend-focused MERN platform • Verified vehicle inspection • Learning-by-building record

1. Day 1 Objective

Establish the first backend foundation of Niyukt and understand the request flow rather than merely copying a working Express example. Day 1 focused on Express application setup, server startup, API versioning, routing, a standardized health response, and the first controller separation.

2. Project State at the End of Day 1

Path	Purpose / current state
Backend/src/app.js	Configures the Express application and JSON middleware; mounts the versioned API router.
Backend/src/server.js	Starts the HTTP server with app.listen() on port 4000.
Backend/src/Controllers/health.controller.js	Contains healthCheck(), the HTTP-level health response handler.
Backend/src/routes/health.routes.js	Defines GET /health and delegates handling to healthCheck.
Frontend/	Created as the frontend area; no frontend work was part of Day 1.
docs/	Engineering documentation set established for the project.

3. Concepts Understood

app.js vs server.js: app.js configures the Express application; server.js starts the HTTP server.

app.listen(): Starts the server so it can accept incoming HTTP requests.

Relative paths: Node resolves require() paths relative to the file doing the import.

app.get() vs app.use(): app.get() registers a GET route; app.use() mounts middleware or routers.

express.Router(): Creates a modular route group that can be mounted into the main Express app.

API versioning: The health router is mounted at /api/v1, so /health becomes /api/v1/health.

Consistent response contracts: Niyukt uses a predictable success/message/data shape instead of random endpoint formats.

Responsibility-based filenames: Names such as health.routes.js and health.controller.js communicate what each file owns.

4. Working Request Flow

GET /api/v1/health → app.js receives the request through the mounted router → health.routes.js matches GET /health → healthCheck controller executes → res.json() sends the response.

Expected response:

```
{
  "success": true,
  "message": "Niyukt API is healthy",
  "data": { "status": "ok" }
}
```

The endpoint was tested successfully in the browser. Requesting /health directly returns 404 because the router is intentionally mounted under /api/v1.

5. Controller Concept Learned

A controller is the HTTP boundary between the incoming request and application logic. Its core job is to receive the request, extract HTTP-specific input, call the appropriate service when business logic is required, and turn the result into a standardized HTTP response.

Responsibilities deliberately kept outside controllers:

- Business rules → services
- Database/data-access operations → repositories
- Authentication/authorization concerns → middleware and appropriate authorization checks
- Routing decisions → routes

Important nuance learned: a service returns the result of a business operation; the controller is the layer that translates that result into an HTTP response.

6. First Refactor Completed

The health endpoint was refactored from having its handler directly in the route to using a dedicated health controller. The route now connects the URL/method to healthCheck, while the controller owns the HTTP response.

Conceptual separation:

health.routes.js → “Which function handles GET /health?”

health.controller.js → “How do I handle this HTTP request and produce the response?”

7. Small Practical Lesson

In PowerShell, **mkdir** creates a directory. **New-Item filename.js** creates a file. The shorter PowerShell alias **ni filename.js** can also be used. A mistaken mkdir command created an accidental folder named health.controlles.js; it was identified and deleted.

8. Documentation System — How to Maintain It

Niyukt should be documented while it is being built. The project blueprint defines dedicated documents for requirements, architecture, database design, API contracts, authentication/authorization, state machine, evidence, testing, performance, security, deployment, decisions, limitations, future roadmap, and interview notes.

For daily work, maintain the logs as living engineering records:

Progress Log — date • goal • built • learned • problem • decision • tests • next step

Learning Log — concept • problem that triggered it • theory • implementation • trade-offs • interview explanation

Bug Log — expected • actual • reproduction • root cause • fix • regression test • lesson

Session Log — what happened today • what must happen next

Performance Log — endpoint • baseline • bottleneck • change • after-result • trade-off

Security Log — risk • scenario • protection • test • result • improvement

Rule for future sessions: when a feature is completed, update the progress record, learning notes, tests, and next step. When a decision changes architecture, record the context, alternatives, choice, trade-offs, and consequences.

9. Day 1 Documentation Entry — Ready to Record

Date: 2026-09-04

Goal: Establish Express backend foundation and understand route/controller separation.

Built: Express app, server startup, /api/v1 routing, health endpoint, health controller.

Learned: app.js/server.js separation, Router, mounting, API versioning, response contracts, controller responsibility.

Problem: Accidentally created a directory named health.controlles.js while learning PowerShell file creation.

Fix: Identified the directory/file distinction, deleted the accidental folder, and verified the controller file.

Tests: GET /api/v1/health returns the expected JSON response; /health returns 404 as expected.

Decision: Keep routes thin and move HTTP handling into controllers. Do not create unnecessary service layers for simple endpoints such as health.

Next step: Learn req.body and build the first small task controller, then introduce the service layer when business logic appears.

10. Day 1 Engineering Principle

Do not build layers because a diagram says they exist. Build a layer when a responsibility belongs there. The goal is separation of concerns that makes the system easier to understand, test, change, and explain.

Niyukt blueprint principle: Build it. Break it. Debug it. Measure it. Explain it. Document it.

Day 1 record prepared from the current Niyukt V1 checkpoint and the project's Engineering Blueprint.